

---

# Státnice: rychlá příprava na trojku

Informatika - Umělá inteligence

zaměření **Strojové učení**

---

Lean studijní text optimalizovaný na **nejméně času k jistému průchodu** (známka 3, >98 %).  
Strukturováno přesně podle oficiálních požadavků (požadavky/detailni-pozadavky.pdf).

## Jak tento text funguje

- Každé téma má dvě části: šedý box **TÉMA** = **oficiální požadavek doslova** (co se může zkoušet), a pod ním *Učivo* = naše stručné, intuitivní vysvětlení.
- **Odpověď podle bodů** ukazuje, co stačí říct na 1 / 2 / 3 body. **Cíl pro průchod je zelená = 2 body.**
- **Vyzkoušej se** = otázky na aktivní vybavování; **Pozor** = častá past.
- Obrázky jsou tam, kde ušetří čas. Text je **přepsaný pro pochopení**, ne opsaný z poznámek.

# Obsah

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Strategie: jak projít s minimem času</b>               | <b>2</b>  |
| 1.1      | Rubrika zvládnutí (sebehodnocení)                         | 2         |
| <b>2</b> | <b>Rozsah: co se učit a co ne</b>                         | <b>2</b>  |
| <b>3</b> | <b>Část I: Společná matematika</b>                        | <b>3</b>  |
| 3.1      | Základy diferenciálního a integrálního počtu              | 3         |
| 3.2      | Algebra a lineární algebra                                | 3         |
| 3.3      | Diskrétní matematika                                      | 4         |
| 3.4      | Teorie grafů                                              | 5         |
| 3.5      | Pravděpodobnost a statistika                              | 6         |
| 3.6      | Logika                                                    | 7         |
| <b>4</b> | <b>Část II: Společná informatika</b>                      | <b>9</b>  |
| 4.1      | Automaty a jazyky                                         | 9         |
| 4.2      | Algoritmy a datové struktury                              | 10        |
| 4.3      | Programovací jazyky                                       | 11        |
| 4.4      | Architektura počítačů a operačních systémů                | 13        |
| <b>5</b> | <b>Část III: Specializace - Základy umělé inteligence</b> | <b>16</b> |
| 5.1      | Řešení úloh prohledáváním                                 | 16        |
| 5.2      | Splňování omezujících podmínek (CSP)                      | 17        |
| 5.3      | Logické uvažování                                         | 18        |
| 5.4      | Pravděpodobnostní uvažování                               | 19        |
| 5.5      | Reprezentace znalostí                                     | 20        |
| 5.6      | Automatické plánování                                     | 21        |
| 5.7      | Markovské rozhodovací procesy (MDP)                       | 22        |
| 5.8      | Hry a teorie her                                          | 23        |
| 5.9      | Strojové učení (přehledově)                               | 24        |
| <b>6</b> | <b>Část IV: Specializace - Strojové učení</b>             | <b>26</b> |
| 6.1      | Učení s učitelem (základní pojmy, evaluace, regularizace) | 26        |
| 6.2      | Učení založené na příkladech - k nejbližším sousedů (kNN) | 27        |
| 6.3      | Lineární regrese                                          | 27        |
| 6.4      | Logistická regrese                                        | 28        |
| 6.5      | Rozhodovací stromy                                        | 29        |
| 6.6      | Metoda podpůrných vektorů (SVM)                           | 30        |
| 6.7      | Kombinace více modelů (ensemble)                          | 31        |
| 6.8      | Statistické testy                                         | 31        |
| 6.9      | Učení bez učitele (shlukování, PCA)                       | 32        |

# 1 Strategie: jak projít s minimem času

## Bodový rozpočet (proč to stačí)

8 otázek po 0-3 bodech. **Průchod (známka 3)** = aspoň 12 bodů celkem **a** aspoň 5 ze společných okruhů (4 otázky) **a** aspoň 7 ze specializace (4 otázky).

- **Specializace je klíčová** (potřebuješ 7/12, tj. průměr 1,75 na otázku). Sem dej většinu času (ML + Základy UI  $\approx 60\%$ ). Společné okruhy mají nízký floor 5/12 ( $\approx 1,25$  na otázku) - stačí breadth, ne hloubka.
- **Hlavní riziko je nula z přidělené otázky**, ne chybějící důkaz. Proto měj u *každého* tématu aspoň „kostku na 1 bod“. Žádné nuly  $\Rightarrow$  pravděpodobnost průchodu jde nad 98 %.
- **Trénuj nad hranici** (cíl  $\sim 16/24$ ), protože  $5+7=12$  nemá rezervu.
- **Vynech** (bezpečně): dlouhé důkazy, plná odvození, jazykové varianty mimo zvolený jazyk, vše mimo rozsah. Detaily viz sekce 2.

## 1.1 Rubrika zvládnutí (sebehodnocení)

| stav          | co umíš                                                  |
|---------------|----------------------------------------------------------|
| • červená (0) | ani definici                                             |
| • žlutá (1)   | definice + pár pojmů                                     |
| • zelená (2)  | definice + algoritmus/věta + příklad + intuice (cíl)     |
| • modrá (3)   | zelená + přesné podmínky/detail (jen u nejdůležitějších) |

Cíl: každé téma aspoň žlutě; specializace  $\sim 85\%$  zeleně; nejdůležitější modely modře. Podrobná strategie a rozpočet času: **STRATEGY.md** v repozitáři.

## 2 Rozsah: co se učit a co ne

Závazný zdroj: [pozadavky/detailni-pozadavky.pdf](#). Pro zaměření **Strojové učení** se zkouší: společná matematika (6 témat), společná informatika (4 témata), a ze specializace *Základy UI* (téma 1) a *Strojové učení* (téma 3).

### Mezera v poznámkách - ověř / doplň

**Záměrně NEzahrnuto** (nezkouší se, neuč se): Robotika a NLP; neuronové sítě a hluboké učení (perceptron, MLP, CNN, word2vec patří k NLP); počítačové sítě, databáze a SQL, grafika, softwarové inženýrství, RSA/FFT a pokročilé algoritmy, výpočetní geometrie.

## 3 Část I: Společná matematika

Nižší priorita: ze společných okruhů (matematika + informatika) stačí 5/12. Cíl je **breadth, ne hloubka** - u každého tématu umět definice a hlavní pojmy (žlutá/zelená), aby z žádné přidělené otázky nebyla nula. Dlouhé důkazy nejsou potřeba.

### 3.1 Základy diferenciálního a integrálního počtu

priorita: střední      cíl: žlutá-zelená (1-2 body)

Posloupnosti a jejich limity (aritmetika limit, věta o dvou polícajtech); řady (geometrická, harmonická); limita funkce a spojitost (nabývání mezhodnot a maxima); derivace (l'Hospital, průběh funkce: extrémy, monotonie, konvexita; Taylorův polynom); integrály (primitivní funkce, Riemannův/Newtonův integrál; aplikace: odhady součtů řad, obsahy, objemy a povrchy rotačních útvarů, délka křivky).

*Učivo (jak na to):*

*Limita* = kam se hodnoty blíží. *Derivace* = okamžitá rychlost změny (sklon tečny). *Integrál* = orientovaná plocha pod grafem. Derivace a integrál jsou navzájem inverzní (základní věta analýzy).

Odpověď podle bodů (cíľ pro průchod: zelená = 2 body)

- 1 **Minimum:**  $\lim a_n = L$ : od jistého  $n$  jsou členy libovolně blízko  $L$ . Derivace  $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h)-f(a)}{h}$ . Riemannův integrál = orientovaná plocha pod grafem.
- 2 **+ k tomu:** *Věta o dvou polícajtech*: sevřená posloupnost má limitu jako její meze. *Geometrická řada*  $\sum q^n = \frac{1}{1-q}$  pro  $|q| < 1$ , harmonická  $\sum \frac{1}{n}$  diverguje. *l'Hospital*: pro typ  $\frac{0}{0}$  nebo  $\frac{\infty}{\infty}$  lze za podmínek věty nahradit podílem derivací. Průběh:  $f' > 0$  roste,  $f'' > 0$  konvexní, kandidáty na extrém dávají body s  $f' = 0$  nebo kde derivace není definovaná. *Newton-Leibniz*:  $\int_a^b f = F(b) - F(a)$ .
- 3 **Bonus:** *Taylor*  $f(x) \approx \sum \frac{f^{(k)}(a)}{k!} (x-a)^k$ . Aplikace integrálu: objem rotačního tělesa  $\pi \int f(x)^2 dx$ , délka křivky  $\int \sqrt{1+f'(x)^2} dx$ , povrch rotační plochy  $2\pi \int f(x) \sqrt{1+f'(x)^2} dx$ . Integrální kritérium: pro nezápornou nerostoucí  $f$  konverguje  $\sum f(k)$  právě když je nevlastní integrál konečný.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Kdy konverguje geometrická řada a jaký má součet?
- Co poznám z  $f'$  a co z  $f''$ ?
- Jak souvisí Riemannův integrál s primitivní funkcí?

**Pozor (častá past):** Harmonická řada diverguje, i když členy jdou k nule. l'Hospital jen na neurčité výrazy.

### 3.2 Algebra a lineární algebra

priorita: střední      cíl: žlutá-zelená (1-2 body)

Algebraické struktury (grupy, permutace, tělesa včetně konečných); soustavy lineárních rovnic (Gaussova a Gauss-Jordanova eliminace); matice (hodnost, regulární a inverzní); vektorové prostory (lineární nezávislost, báze, dimenze, Steinitzova věta, podprostory); lineární zobrazení (obraz, jádro, isomorfismus); skalární součin (norma, Cauchy-Schwarz, ortonormální báze, Gram-Schmidt, ortogonální projekce a matice); determinanty (Laplaceův rozvoj, geometrie); vlastní čísla a vektory (charakteristický polynom, diagonalizace, spektrální rozklad, symetrické matice); pozitivně (semi)definitní matice (Choleského rozklad).

*Učivo (jak na to):*

Lineární algebra studuje *vektorové prostory* a *lineární zobrazení* (= násobení maticí). Hodně pojmů jsou jen různé pohledy na totéž: hodnost, dimenze obrazu, počet pivotů.

**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

**1 Minimum:** *Báze* = lineárně nezávislá množina, která generuje prostor; její velikost = *dimenze*. *Hodnost* matice = počet pivotů (= dimenze obrazu). Matice je *regulární*  $\iff$  má inverzi  $\iff \det \neq 0 \iff$  plná hodnost.

**2 + k tomu:** *Gaussova eliminace* řeší soustavy i počítá hodnost/inverzi. *Jádro* = řešení  $Ax = 0$ ; platí  $\dim \ker + \text{rank} = n$ . *Vlastní číslo*:  $Ax = \lambda x$  ( $x \neq 0$ ), kořeny char. polynomu  $\det(A - \lambda I) = 0$ . *Skalární součin* dává normu a kolmost; *Gram-Schmidt* vyrobí ortonormální bázi. Souřadnice vektoru vůči ortonormální bázi jsou *Fourierovy koeficienty*  $\langle u | v_i \rangle$  (a *ortogonální projekce* na podprostor je součet  $\sum_i \langle u | v_i \rangle v_i$ ). Pythagorova věta a Cauchy-Schwarzova nerovnost  $|\langle u | v \rangle| \leq \|u\| \|v\|$ .

**3 Bonus:** *Diagonalizace*  $A = S \Lambda S^{-1}$  (sloupce  $S$  = vlastní vektory). Symetrické matice mají reálná vl. čísla a ortogonální diagonalizaci  $A = Q \Lambda Q^T$ . Reálná symetrická matice je *pozitivně definitní* ( $x^T A x > 0$  pro  $x \neq 0$ )  $\iff$  má kladná vl. čísla  $\iff$  existuje Choleského rozklad  $A = LL^T$ . Grupa má asociativní operaci, neutrální a inverzní prvky; těleso má sčítání a násobení kompatibilní distributivitou, konečné těleso má řád mocniny prvočísla. Permutace se skládá z cyklů, rozkládá na transpozice a má znaménko. Souřadnice a matice lineárního zobrazení závisí na bázi; matice přechodu je regulární. Ortogonální matice zachovává skalární součin a délky. Determinant měří orientovaný objem, je multiplikativní a stopa je součet vlastních čísel.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co znamená, že je matice regulární (vyjmenuj ekvivalentní podmínky)?
- Jak najdu vlastní čísla a co je vlastní vektor?
- Co je pozitivně definitní matice a jak souvisí s vlastními čísly?

**Pozor (častá past):** Vlastní čísla = kořeny char. polynomu, ne prvky na diagonále (pokud matice není trojúhelníková).

### 3.3 Diskrétní matematika

**priorita: střední** **cíl: žlutá-zelená (1-2 body)**

Relace a jejich vlastnosti (reflexivita, symetrie, antisymetrie, tranzitivita); ekvivalence a rozkladové třídy; částečná uspořádání (řetězec, antiřetězec, výška a šířka a věta o jejich vztahu); funkce (prostá, na, bijekce; počty funkcí mezi konečnými množinami); permutace; kombinační čísla a binomická věta; princip inkluze a exkluze (důkaz a aplikace); Hallova

věta o systému různých reprezentantů (vztah k párování v bipartitním grafu, polynomiální algoritmus).

*Učivo (jak na to):*

Hodně z toho je „počítání možností“ a struktura na konečných množinách. *Inkluze-exkluze* opravuje dvojí započítání; *Hallova věta* říká, kdy lze každého spárovat.

**Odpověď podle bodů (cíl pro průchod: zelená = 2 body)**

- ❶ **Minimum:** *Ekvivalence* = reflexivní + symetrická + tranzitivní; rozkládá množinu na třídy. *Bijekce* = prostá i na. Počet funkcí  $A \rightarrow B$  je  $|B|^{|A|}$ .
- ❷ **+ k tomu:** *Částečné uspořádání*: řetězec (po dvou porovnatelné), antiřetězec (žádné dva); *Dilworthova věta* dává vztah šířky a pokrytí řetězci. *Inkluze-exkluze*:  $|A \cup B| = |A| + |B| - |A \cap B|$  (obecně se střídají znaménka). *Binomická věta*:  $(a + b)^n = \sum \binom{n}{k} a^k b^{n-k}$  (kombinační čísla = Pascalův trojúhelník).
- ❸ **Bonus:** *Hallova věta*: bipartitní graf má párování pokrývající  $A \iff$  pro každou  $X \subseteq A$  platí  $|N(X)| \geq |X|$  (Hallova podmínka); související *Kőnigova věta*: max. párování = min. vrcholové pokrytí v bipartitním grafu. Aplikace inkluze-exkluze: počet surjekcí, Eulerova funkce, problém šatnářky. Počet prostých funkcí  $N \rightarrow M$  je  $m^n$  a bijekcí na  $n$  prvcích je  $n!$ . V konečné neprázdné ČUM existují minimální a maximální prvky, výška krát šířka je aspoň  $|X|$ .

*Vyzkoušej se (zakryj text a odpověz nahlas)*

- Jaké tři vlastnosti dělají z relace ekvivalenci?
- Jak zní Hallova podmínka?
- Kolik je funkcí (a kolik bijekcí) mezi konečnými množinami?

**Pozor (častá past):** Antisymetrie neznamená „není symetrická“. Bijekce mezi  $A, B$  existuje jen při  $|A| = |B|$ .

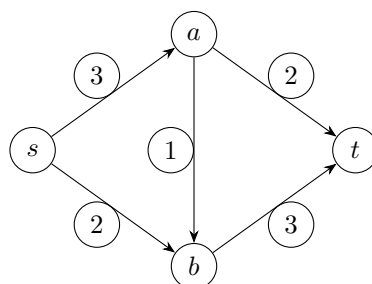
### 3.4 Teorie grafů

**priorita: střední**   **cíl: zelená (2 body), vděčné téma**

Základní pojmy (izomorfismus, stupeň vrcholu, doplněk, bipartitní graf); příklady (úplný, cesty, kružnice); souvislost, komponenty, vzdálenost; stromy (charakteristiky); rovinné grafy (Eulerova formule a max. počet hran); barevnost (vztah barevnosti a klikovosti); hranová a vrcholová souvislost (Mengerova věta); orientované grafy (silná a slabá souvislost); toky v sítích (definice, existence max. toku, Ford-Fulkerson pro celočíselné kapacity).

*Učivo (jak na to):*

Grafy = vrcholy a hrany. Mnoho úloh = „dá se projít / spojit / obarvit / protlačit tok?“.



Sít s kapacitami. Max tok = min řez (zde 5).

**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

- ➊ **Minimum:** *Strom* = souvislý graf bez kružnic, na  $n$  vrcholech má  $n - 1$  hran. *Bipartitní* = vrcholy lze obarvit 2 barvami (žádná hrana uvnitř části)  $\iff$  bez liché kružnice.
- ➋ **+ k tomu:** *Rovinný graf:* *Eulerova formule*  $v - e + f = 2$ ; pro jednoduchý rovinný graf s  $v \geq 3$  plyne max.  $3v - 6$  hran. *Barevnost*  $\geq$  klikovost. *Tok:* respektuje kapacity a zachování; *Ford-Fulkerson* = opakuji zlepšující cesty v reziduální síti; max tok = min řez.
- ➌ **Bonus:** *Mengerova věta:* pro dva vrcholy je maximum hranově (vrcholově) disjunktních cest rovno velikosti minimálního hranového (vrcholového) separátoru. Orientovaný graf: silně souvislý = cesta mezi každými dvěma vrcholy v obou směrech, slabě souvislý = souvislý podkladový graf. Eulerovský graf je souvislý a všechny stupně jsou sudé; v orientovaném grafu potřebuje vyvážené vstupní a výstupní stupně. Kuratowski: nerovinný graf obsahuje dělení  $K_5$  nebo  $K_{3,3}$ ; rovinný graf bez trojúhelníků má nejvýše  $2v - 4$  hran.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Kolik hran má strom a jak poznám bipartitní graf?
- Co říká Eulerova formule a kolik nejvíc hran má rovinný graf?
- Jak funguje Ford-Fulkerson a co je max-flow = min-cut?

**Pozor (častá past):** Eulerova formule platí pro souvislý rovinný graf; odhad  $3v - 6$  navíc předpokládá jednoduchost a  $v \geq 3$ . Bipartitní  $\iff$  bez liché kružnice.

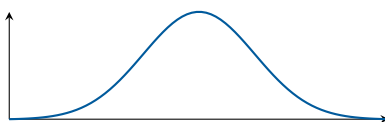
### 3.5 Pravděpodobnost a statistika

**priorita: střední** **cíl: žlutá-zelená (1-2 body)**

Pravděpodobnostní prostor, jevy, nezávislost, podmíněná pravděpodobnost, Bayesův vzorec; náhodné veličiny (diskrétní i spojité), distribuční funkce, hustota/pravděpodobnostní funkce; střední hodnota (linearita, Markovova nerovnost); rozptyl (vzorec pro součet); konkrétní rozdělení (geometrické, binomické, Poissonovo, normální, exponenciální); limitní věty (zákon velkých čísel, centrální limitní věta); bodové a intervalové odhady; testování hypotéz (chyby 1. a 2. druhu, hladina významnosti).

*Učivo (jak na to):*

Pravděpodobnost popisuje nejistotu. *Střední hodnota* = dlouhodobý průměr, *rozptyl* = míra rozkolísanosti. *CLV* říká, proč se všude objevuje normální rozdělení.



Normální rozdělení (Gaussova křivka): limita součtu mnoha nezávislých vlivů (CLV).

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 **Minimum:** Podmíněná  $P(A|B) = \frac{P(A \cap B)}{P(B)}$ . Bayes  $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ . Střední hodnota  $E[X] = \sum x_i p_i$  (resp.  $\int x f(x)$ ).

2 + k tomu: Linearita  $E[X + Y] = E[X] + E[Y]$  (vždy). Rozptyl  $\text{var}(X) = E[X^2] - E[X]^2$ ; obecně  $\text{var}(X + Y) = \text{var}X + \text{var}Y + 2\text{cov}(X, Y)$ , pro nezávislé kovariance zmizí. Markovova nerovnost: pro  $X \geq 0$  a  $a > 0$  platí  $P(X \geq a) \leq E[X]/a$ .

3 **Bonus:** Zákon velkých čísel: průměr  $\rightarrow E[X]$ . CLV: normovaný součet mnoha nezávislých veličin má přibližně normální rozdělení. Test hypotéz: chyba 1. druhu = zamítnu platnou  $H_0$  (hladina  $\alpha$ ), 2. druhu = nezamítnu neplatnou. Konkrétní rozdělení: Bernoulli, geometrické, binomické, Poissonovo, uniformní, exponenciální a normální. Bodové odhady řeší nestrannost, konzistenci, bias a MSE; tvoří se metodou momentů nebo MLE. Konfidenční interval má spolehlivost  $1 - \alpha$ , p-hodnota je nejmenší hladina, na níž zamítáme.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak zní Bayesův vzorec?
- Kdy platí  $\text{var}(X + Y) = \text{var}X + \text{var}Y$ ?
- Co říká centrální limitní věta?

**Pozor (častá past):** Linearita střední hodnoty platí vždy, ale součet rozptylů jen pro nezávislé veličiny.

### 3.6 Logika

priorita: střední      cíl: žlutá-zelená (1-2 body)

Syntaxe (výroková a predikátová logika, normální tvary CNF/DNF/PNF, převody, použití pro SAT a rezoluci); sémantika (model teorie, splnitelnost, tautologie, důsledek); extenze teorií (konzervativnost, skolemizace); dokazatelnost (formální důkaz, dokazovací systémy: tablo, rezoluce, příp. Hilbertovský kalkul); věty o kompaktnosti a úplnosti; rozhodnutelnost.

Učivo (jak na to):

Syntaxe = forma formulí, sémantika = jejich pravdivost v modelech. Dokazatelnost (syntaktická) a platnost (sémantická) spojuje věta o úplnosti.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 **Minimum:** Tautologie = pravdivá ve všech ohodnoceních; splnitelná = pravdivá aspoň v jednom.  $T \models \varphi$ :  $\varphi$  platí v každém modelu  $T$ . CNF = konjunkce klauzulí, DNF = disjunkce konjunkcí.

2 + k tomu: Rezoluce (na CNF): z klauzulí  $C \vee \ell$  a  $D \vee \neg \ell$  odvodí  $C \vee D$ ; spor = prázdná klauzule (důkaz nesplnitelnosti). Tablo = systematické hledání modelu. Věta o úplnosti:  $T \models \varphi \iff T \vdash \varphi$ .

3 **Bonus:** Kompaktnost: teorie má model  $\iff$  každá konečná část má model. Skolemizace odstraní existenční kvantifikátory. Rozhodnutelnost: výroková logika je rozhodnutelná (SAT), predikátová obecně ne; rekurzivně axiomatizovaná kompletní teorie je rozhodnutelná, ale např. Peanova aritmetika kompletní není. PNF má kvantifikátory v prefixu; Horn-SAT je lineární přes jednotkovou propagaci. Extenze je konzervativní, když nepřidá nové důsledky ve starém jazyce. Tablo je sporné, když se uzavřou všechny větve; korektnost a úplnost spojují důkaz s platností. Gödel: dost silná bezesporná rekurzivně axiomatizovaná aritmetika je nekompletní.



Vyzkoušej se (zakryj text a odpověz nahlas)

- Rozdíl mezi tautologií, splnitelností a důsledkem?
- Jak funguje rezoluční pravidlo a co dokazuje prázdná klauzule?
- Co říká věta o úplnosti?

**Pozor (častá past):** Nezaměňuj „splnitelná“ (aspoň jeden model) a „tautologie“ (všechny modely). Rezoluce dokazuje nesplnitelnost (hledá spor).

## 4 Část II: Společná informatika

Společná informatika je floor pro společné okruhy: cíl je umět každé téma bez nuly a rychle poskládat zelenou odpověď. Tady stačí kostra, definice, jeden příklad a hlavní pasti.

### 4.1 Automaty a jazyky

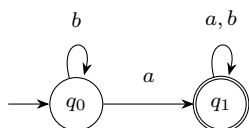
priorita: střední    cíl: zelená (2 body)

#### Automaty a jazyky

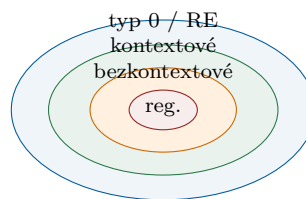
- Regulární jazyky
- regulární gramatiky
- deterministický a nedeterministický konečný automat
- regulární výrazy
- Bezkontextové jazyky
- bezkontextové gramatiky, jazyk generovaný gramatikou
- zásobníkový automat, třída jazyků přijímaných zásobníkovými automaty
- Rekurzivně spočetné jazyky
- gramatika typu 0
- Turingův stroj
- algoritmicky nerozhodnutelné problémy
- Chomského hierarchie
- schopnost zařazení konkrétního jazyka do Chomského hierarchie (zpravidla sestavení odpovídajícího automatu či gramatiky)

*Učivo (jak na to):*

Jazyk je množina slov. Čím silnější model výpočtu dovolím, tím složitější jazyky umím popsat: regulární přes konečnou paměť, bezkontextové přes zásobník, rekurzivně spočetné přes Turingův stroj.



DFA pro slova obsahující aspoň jedno  $a$ .



Chomského hierarchie jako vnořené třídy.

**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

- 1 Minimum:** Regulární jazyk pozná konečný automat a popíše regulární výraz nebo regulární gramatika. DFA má jeden přechod pro stav a znak, NFA může mít více přechodů nebo  $\varepsilon$ -kroky, ale poznává stejnou třídu jazyků.
- 2 + k tomu:** Bezkontextový jazyk generuje CFG s pravidly  $A \rightarrow \alpha$  a přijímá ho zásobníkový automat. Zásobník dá paměť pro vnoření, například  $\{a^n b^n \mid n \geq 0\}$ , ale ne pro dvě nezávislá počítání jako  $a^n b^n c^n$ .
- 3 Bonus:** Typ 0 gramatiky odpovídají rekurzivně spočetným jazykům a Turingovým strojům. Existují algoritmicky nerozhodnutelné problémy, klasicky problém zastavení. U zařazování jazyka zkus sestavit automat nebo gramatiku, případně říct, proč nižší třída nestačí. Pumping lemma a Myhill-Nerodeova věta slouží k důkazu neregularity; pro CFL existuje pumping lemma

a Chomského normální tvar. PDA přijímá koncovým stavem nebo prázdným zásobníkem, obě varianty jsou ekvivalentní. Regulární jazyky jsou uzavřené na doplněk, průnik a sjednocení, CFL ne na průnik obecně.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Čím se liší DFA a NFA a proč mají stejnou vyjadřovací sílu?
- Jaký automat přijímá bezkontextové jazyky a k čemu je zásobník?
- Kam v hierarchii patří regulární, bezkontextové a rekurzivně spočetné jazyky?
- Uveď příklad nerozhodnutelného problému.

**Pozor (častá past):** Neříkej, že NFA je silnější než DFA. Je pohodlnější, ale pro konečné automaty popisuje stejnou třídu jazyků.

## 4.2 Algoritmy a datové struktury

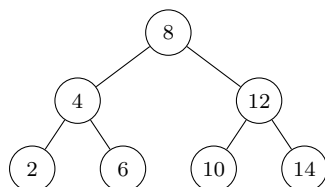
priorita: střední    cíl: zelená (2 body)

### Algoritmy a datové struktury

- Časová složitost algoritmů
- časová a prostorová složitost algoritmu
- měření velikosti dat
- složitost v nejlepším, nejhorším a průměrném případě
- asymptotická notace
- Třídy složitosti
- třídy P a NP
- převoditelnost problémů, NP-těžkost a NP-úplnost
- příklady NP-úplných problémů a převodů mezi nimi
- Metoda rozděl a panuj
- princip rekurzivního dělení problému na podproblémy
- výpočet složitosti pomocí rekurentních rovnic
- Master theorem (kuchařková věta) (bez důkazu)
- aplikace
- Mergesort
- násobení dlouhých čísel
- Binární vyhledávací stromy
- definice vyhledávacího stromu
- operace s nevyvažovanými stromy
- AVL stromy (definice)
- Třídění
- primitivní třídící algoritmy (Bubblesort, Insertsort)
- Quicksort
- dolní odhad složitosti porovnávacích třídících algoritmů
- Grafové algoritmy
- prohledávání do šířky a do hloubky
- topologické třídění orientovaných grafů
- nejkratší cesty v ohodnocených grafech (Dijkstrův a Bellmanův-Fordův algoritmus)
- minimální kostra grafu (Jarníkův a Borůvkův algoritmus)
- toky v sítích (Ford-Fulkerson algoritmus)

*Učivo (jak na to):*

U algoritmů se zkouší hlavně rozpoznat správnou techniku, říct invariant nebo datovou strukturu a umět odhadnout složitost. U společné části stačí mít mapu pojmů a typické časy v ruce.



BST: vlevo menší, vpravo větší

AVL: v každém uzlu rozdíl výšek nejvýše 1

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

**1 Minimum:** Složitost měř vzhledem k velikosti vstupu a rozliš čas, prostor, nejlepší, průměrný a nejhorší případ. Značení  $O$ ,  $\Omega$ ,  $\Theta$  popisuje asymptotický růst. P je řešitelné v polynomiálním čase, NP má polynomiálně ověřitelné certifikáty.

**2 + k tomu:** NP-těžkost se ukazuje polynomiálním převodem ze známého NP-těžkého problému, NP-úplný problém je v NP a je NP-těžký. Rozděl a panuj: rozděl, vyřeš podproblémy, slož výsledek, například mergesort  $T(n) = 2T(n/2) + O(n) = O(n \log n)$ . BST hledá podle pořadí klíčů, AVL drží vyvážení rotacemi. Porovnávací třídění má dolní odhad  $\Omega(n \log n)$ .

**3 Bonus:** Quicksort má průměrně  $O(n \log n)$  a nejhůře  $O(n^2)$ . BFS je fronta a vzdálenosti v neohodnoceném grafu, DFS je zásobník a časy průchodu. Topologické třídění funguje pro DAG. Dijkstra vyžaduje nezáporné hrany, Bellman-Ford zvládá záporné hrany, Jarník a Borůvka řeší MST, Ford-Fulkerson zlepšuje tok po reziduálních cestách. Master theorem pro  $T(n) = aT(n/b) + \Theta(n^c)$  porovnává  $a/b^c$ ; Karacuba násobí dlouhá čísla v  $O(n^{\log_2 3})$ . NP převody jsou reflexivní a tranzitivní; typické NP-úplné problémy jsou SAT, 3-SAT, klika, nezávislá množina,  $k$ -obarvitelnost, Hamiltonovská kružnice, batoh a součet podmnožiny. AVL má hloubku  $\Theta(\log n)$ , porovnávací třídění má dolní mez  $\Omega(n \log n)$ .

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co musí platit, aby byl problém NP-úplný?
- Jak spočítáš mergesort přes rekurentní rovnici?
- Jaký je rozdíl mezi BST a AVL stromem?
- Kdy použiješ Dijkstru a kdy Bellman-Forda?

**Pozor (častá past):** NP-těžký problém nemusí být v NP. NP-úplnost vyžaduje obojí: patří do NP a každý problém z NP se na něj polynomiálně převede.

### 4.3 Programovací jazyky

priorita: střední    cíl: zelená (2 body)

#### Programovací jazyky

Některé následující body definují varianty požadavků pro různé individuální volby povinně volitelných předmětů. Vyžaduje se zvládnutí všech bodů bez označení  $\nabla$  a zvládnutí všech

bodů s označením  $\forall$  pro jeden z jazyků C#, C++ nebo Java.

- Koncepty pro abstrakci, zapouzdření a polymorfismus.
- související konstrukty programovacích jazyků
- třídy, rozhraní, metody, datové položky, dědičnost, viditelnost
- (dynamický) polymorfismus, statické a dynamické typování
- jednoduchá dědičnost
- $\forall$  virtuální a nevirtuální metody v C++ a C#
- vícenásobná dědičnost a její problémy
- $\forall$  interfaces v C#
- implementace rozhraní (interface)
- vhodné použití uvedených konceptů
- Primitivní a objektové typy a jejich reprezentace.
- číselné a výčtové typy
- $\forall$  hodnotové a referenční typy v C#
- $\forall$  reference, imutabilní typy a boxing v C# a Javě
- Generické typy a funkcionální prvky (procedurálních programovacích jazyků).
- $\forall$  generické typy v Javě a C# (bez omezení typových parametrů)
- $\forall$  typy reprezentující funkce v C++, C#, nebo Javě
- lambda funkce a funkcionální rozhraní
- Manipulace se zdroji a mechanismy pro ošetření chyb.
- správa životního cyklu zdrojů v případě výskytu chyb
- $\forall$  using v C#
- konstrukce pro obsluhu a propagaci výjimek
- Životní cyklus objektů a správa paměti.
- alokace (alokace statická, na zásobníku, na haldě)
- inicializace (konstruktory, volání zděděných konstruktorů)
- destrukce (destruktory, finalizátory)
- explicitní uvolňování objektů, reference counting, garbage collector
- Vlákna a podpora synchronizace.
- reprezentace vláken v programovacích jazycích
- specifikace funkce vykonávané vláknem a základní operace na vlákny
- časově závislé chyby a mechanismy pro synchronizaci vláken
- Implementace základních prvků objektových jazyků.
- základní objektové koncepty v konkrétním jazyce
- implementace a interní reprezentace primitivních typů
- implementace a interní reprezentace složených typů a objektů
- implementace dynamického polymorfismu (tabulka virtuálních metod)
- Nativní a interpretovaný běh, řízení překladač a sestavení programu.
- reprezentace programu, bytecode, interpret jazyka
- just-in-time (JIT) a ahead-of-time (AOT) překlad
- proces sestavení programu, oddělený překlad, linkování
- staticky a dynamicky linkované knihovny
- běhové prostředí procesu a vazba na operační systém

*Učivo (jak na to):*

Drž se C#. Odpověď stav na třech vrstvách: jazykové konstrukty, běhová reprezentace a bezpečné zacházení se zdroji a chybami.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

❶ **Minimum:** Abstrakce skrývá detail, zapouzdření chrání stav přes viditelnost, polymorfismus dovolí volat stejnou operaci nad různými typy. V C# jsou třídy, rozhraní, metody, položky, jednoduchá dědičnost tříd a vícenásobná implementace rozhraní. Typování je statické, dynamická vazba se používá u virtuálních metod.

❷ **+ k tomu:** Hodnotové typy se typicky ukládají přímo a kopírují hodnotu, referenční typy se obsluhují přes referenci na objekt na haldě. Boxing zabalí hodnotový typ do objektu. Generika v C# parametrizují třídy a metody typem, delegáti a lambda výrazy reprezentují funkce. Výjimky se propagují zásobníkem, `using` zajistí `Dispose` i při chybě.

❸ **Bonus:** Konstruktor inicializuje objekt a volá konstruktor předka, správu paměti řeší garbage collector, finalizátor není náhrada za uvolnění externího zdroje. Dynamický polymorfismus bývá přes tabulku virtuálních metod. C# běží nad CLR, program se překládá do IL, typicky JIT do strojového kódu, knihovny mohou být staticky nebo dynamicky navázané. Vlákna sdílejí paměť, proto hrozí race conditions a používají se zámky. Diamantový problém řeší C# vícenásobnou implementací rozhraní místo vícenásobné dědičnosti tříd. `ref`, `out` a `in` jsou tracking reference pro hodnotové typy; `readonly` a schovaný setter dávají imutabilitu. RAII v C++ váže zdroj na život objektu; `using` je `try/finally` s `Dispose`. Návrhové vzory: adapter převádí rozhraní, decorator obaluje objekt, factory vyrábí instance, visitor přesouvá operaci mimo hierarchii tříd.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaký je rozdíl mezi hodnotovým a referenčním typem v C#?
- Co v C# znamená `virtual`, `override` a `interface`?
- Kdy použiješ `using` a proč nestačí garbage collector?
- Jak spolu souvisí IL, CLR a JIT?

**Pozor (častá past):** V C# není vícenásobná dědičnost tříd. Vícenásobně lze implementovat rozhraní.

## 4.4 Architektura počítačů a operačních systémů

priorita: střední    cíl: zelená (2 body)

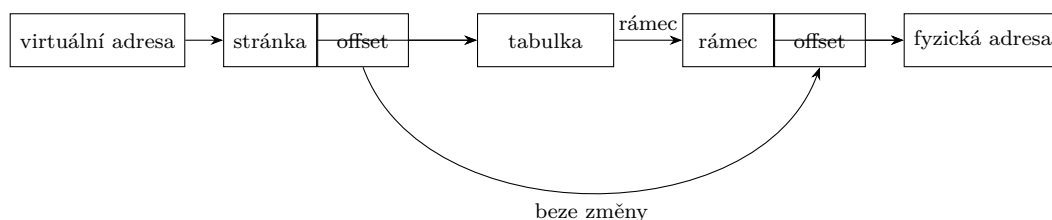
### Architektura počítačů a operačních systémů

- Základní architektura počítače, reprezentace čísel, dat a programů.
- reprezentace a přístup k datům v paměti, adresa, adresový prostor
- ukládání jednoduchých a složených datových typů
- základní aritmetické a logické operace
- Instrukční sada, vazba na prvky vyšších programovacích jazyků.
- Implementovat běžné programové konstrukce vyšších jazyků (přiřazení, podmínka, cyklus, volání funkce) pomocí instrukcí procesoru
- Zapsat běžnou konstrukci vyššího jazyka (přiřazení, podmínka, cyklus, volání funkce), která odpovídá zadané sekvenci (vysvětlených) instrukcí procesoru
- Podpora pro běh operačního systému.
- privilegovaný a neprivilegovaný režim procesoru
- jádro operačního systému
- Rozhraní periferních zařízení a jejich obsluha.
- Popsat roli řadiče zařízení při programem řízené obsluze zařízení (PIO), pro zadané adresy a funkce vstupních a výstupních portů implementovat programem řízenou obsluhu zadaného zařízení (myš, disk)

- Popsat roli přerušení při programem řízené obsluze zařízení (PIO), na úrovni vykonávání instrukcí popsat reakci procesoru (hardware) a operačního systému (software) na žádost o přerušení
- Základní abstrakce, rozhraní a mechanismy OS pro běh programů, sdílení prostředků a vstup/výstup.
- neprivilegované (uživatelské) procesy
- sdílení procesoru
- procesy, vlákna, kontext procesu a vlákna
- přepínání kontextu, kooperativní a preemptivní multitasking
- plánování běhu procesů a vláken, stavy vláken
- sdílení paměti
- Vysvětlit rozdíl mezi virtuální a fyzickou adresou a identifikovat, zda se v zadaném kontextu či fragmentu kódu používá virtuální nebo fyzická adresa
- Na zadaném příkladu identifikovat a vysvětlit význam komponent virtuální a fyzické adresy (číslo stránky, číslo rámce, offset)
- Pro konkrétní adresy a obsah jednoúrovňové stránkovací tabulky řešit úlohy překladu adres
- Vysvětlit roli virtuálních adresových prostorů v ochraně paměti procesů a vláken
- sdílení úložného prostoru
- soubory, analogie s adresovým prostorem
- abstrakce a rozhraní pro práci se soubory
- Paralelismus, vlákna a rozhraní pro jejich správu, synchronizace vláken.
- časově závislé chyby (race conditions)
- kritická sekce, vzájemné vyloučení
- základní synchronizační primitiva, jejich rozhraní a použití
- zámky
- aktivní a pasivní čekání

*Učivo (jak na to):*

Procesor vykonává instrukce nad registry a pamětí, OS odděluje uživatelský kód od jádra a dává procesům iluze: vlastní CPU, vlastní adresový prostor a soubory jako pojmenované úložiště.



**Odpověď podle bodů (cíl pro průchod: zelená = 2 body)**

**1 Minimum:** Počítač má CPU, registry, paměť a zařízení. Data i program jsou v paměti jako bity, adresa určuje místo v adresovém prostoru. Instrukce realizují přiřazení, větvení, cyklus a volání funkce přes čtení, zápis, skoky a práci se zásobníkem. OS běží v privilegovaném režimu, procesy běží uživatelsky.

**2 + k tomu:** Řadič zařízení vystavuje porty nebo registry. PIO znamená, že procesor programově čte a zapisuje stav a data zařízení. Přerušení dovolí zařízení vyžádat obsluhu: hardware uloží minimální stav, skočí na obsluhu v jádře, OS uloží zbytek kontextu a po obsluze se pokračuje. Proces má vlastní adresový prostor a zdroje, vlákno je běžící tok s vlastním kontextem. Preemptivní multitasking přepíná OS, kooperativní čeká na dobrovolné předání.

**3 Bonus:** Virtuální adresa se dělí na číslo stránky a offset, tabulka stránek mapuje stránku na rámec, offset se přenesení. Virtuální adresové prostory chrání procesy mezi sebou. Soubory

jsou pojmenované úložiště s rozhraním open, read, write, close. Race condition vzniká při závislosti na proložení vláken, kritickou sekci chrání vzájemné vyloučení, typicky zámek. Aktivní čekání točí CPU, pasivní čekání vlákno uspí. Harvardská architektura odděluje paměť kódu a dat, von Neumann má jednu paměť. Čísla se ukládají binárně, záporná typicky dvojkovým doplňkem, float jako znaménko, mantisa a exponent; endianita určuje pořadí bytů a zarovnání vkládá mezery do struktur. Syscall je řízený přechod do jádra; DMA přenáší data zařízení bez kopírování CPU. MMU překládá stránky a page fault řeší chybějící mapování. Scheduler používá stavy created, ready, running, blocked, terminated; algoritmy zahrnují FCFS, round robin, multilevel feedback queue a CFS. Semafor má čítač, mutex je binární zámek, barrier čeká na více vláken.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak instrukcemi vyjádříš podmínku, cyklus a volání funkce?
- Co se stane při přerušení z pohledu procesoru a OS?
- Jak přeložíš virtuální adresu přes jednoúrovňovou stránkovací tabulku?
- Jaký je rozdíl mezi procesem, vláknem a jejich kontextem?
- Čím se liší aktivní a pasivní čekání?

**Pozor (častá past):** Virtuální adresa není fyzické místo v RAM. Je to adresa v prostoru procesu, kterou MMU a tabulka stránek překládají na fyzický rámec.



## 5 Část III: Specializace - Základy umělé inteligence

### 5.1 Řešení úloh prohledáváním

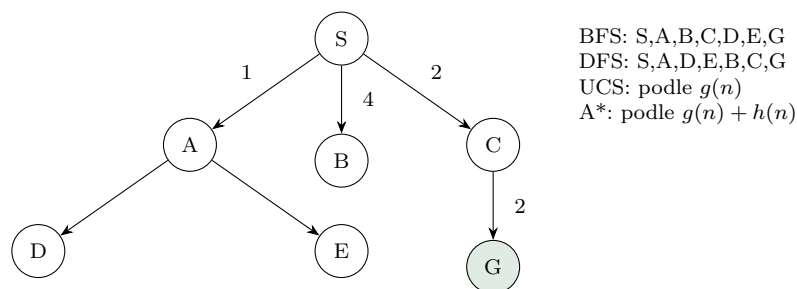
priorita: vysoká    cíl: zelená (2 body)

#### Řešení úloh prohledáváním

- formulace úlohy
- stromové vs. grafové prohledávání
- neinformované prohledávání (DFS, BFS, uniform-cost search)
- informované prohledávání (algoritmus A\*, přípustné a konzistentní heuristiky)

Učivo (jak na to):

Prohledávání je vhodné, když stav umíme brát jako uzel grafu a akci jako hranu. Odpověď u zkoušky má vždy říct: jak problém formuluji, jak držím frontier, podle čeho vybírám další uzel a kdy je řešení optimální.



Úloha je dána počátečním stavem, množinou akcí nebo přechodovým modelem, testem cíle a cenou cesty. Stromové prohledávání si pamatuje cesty ve stromu možností, grafové prohledávání navíc uzavírá již navštívené stavy, aby se netočilo v cyklech.

- **BFS:** fronta, bere nejmenší uzel. Je úplné pro konečné větvení a optimální při stejných cenách kroků.
- **DFS:** zásobník nebo rekurze, jde do hloubky. Má malou paměť, ale není obecně optimální a ve stromové verzi se může zacyklit.
- **Uniform-cost search:** prioritní fronta podle dosavadní ceny  $g(n)$ . Je Dijkstra nad stavovým prostorem, optimální pro nezáporné ceny.
- **A\*:** vybírá minimum  $f(n) = g(n) + h(n)$ . Přípustná heuristika nikdy nepřeceňuje cenu do cíle. Konzistentní heuristika splňuje  $h(n) \leq c(n, a, n') + h(n')$  a dává optimalitu i v grafovém prohledávání bez znovutevívání uzlů.

► **Klíč:** BFS = hloubka, DFS = paměť, UCS = skutečná cena, A\* = skutečná cena plus odhad.

Odpověď podle bodů (cíľ pro průchod: zelená = 2 body)

- 1 **Minimum:** definuj stav, akce, počáteční stav, test cíle a cenu. Vysvětli rozdíl tree search vs. graph search a pojmenuj DFS, BFS, UCS, A\*.
- 2 + k tomu: u každého algoritmu řekni datovou strukturu frontier a optimálnost: BFS pro jednotkové ceny, UCS pro nezáporné ceny, A\* pro přípustnou heuristiku ve stromu a

konzistentní heuristiku v grafu.

**3 Bonus:** složitosti BFS  $O(b^{d+1})$  čas i paměť, DFS čas  $O(b^m)$  a paměť  $O(bm)$  nebo  $O(m)$  při backtrackingu. Dominance heuristiky: větší přípustná heuristika typicky expanduje méně uzlů. Agent může být reflexní, modelový nebo cílový; atomické stavy se hodí pro prohledávání, faktorizované pro CSP a plánování, strukturované pro predikátovou logiku. Greedy best-first search používá jen  $h(n)$  a není obecně optimální ani úplný.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jakých pět částí má formulace problému prohledáváním?
- Kdy je BFS optimální a kdy nestačí?
- Jaký je rozdíl mezi přípustnou a konzistentní heuristikou?
- Co se stane, když DFS pustíš jako tree search na graf s cyklem?

**Pozor (častá past):** Neříkej, že  $A^*$  je vždy optimální. Potřebuje správnou heuristiku a u grafového prohledávání se typicky požaduje konzistence.

## 5.2 Splňování omezujících podmínek (CSP)

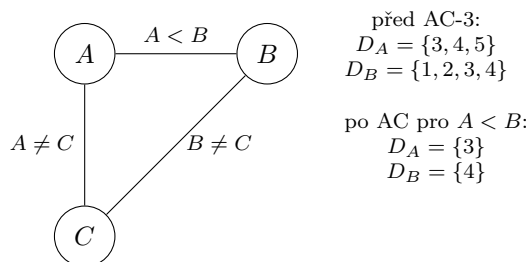
priorita: vysoká    cíl: zelená (2 body)

### Splňování omezujících podmínek

- problém splňování podmínek
- hranová konzistence (algoritmus AC-3)
- algoritmus hledání řešení (MAC)

Učivo (jak na to):

CSP neřeší cestu akcí, ale tabulku hodnot: každé proměnné chceme dát hodnotu tak, aby všechny podmínky seděly. Velký zisk je škrtat nemožné hodnoty dřív, než rozvětvíme backtracking.



CSP má proměnné  $X_i$ , domény  $D_i$  a podmínky jako relace na proměnných. Řešení je úplné přiřazení hodnot, které splní všechny podmínky. Hrana  $(X_i, X_j)$  je hranově konzistentní, když pro každou hodnotu  $x \in D_i$  existuje hodnota  $y \in D_j$ , která splní binární podmínku mezi  $X_i$  a  $X_j$ .

**AC-3:** vlož všechny orientované hrany do fronty. Vyber  $(X_i, X_j)$  a smaž z  $D_i$  hodnoty bez podpory v  $D_j$ . Pokud se  $D_i$  změnila, vrať do fronty všechny hrany  $(X_k, X_i)$  kromě právě opačné. Prázdná doména znamená neúspěch. Jinak dostaneš lokálně vyčištěný CSP, ne nutně hotové řešení.

**MAC:** backtracking nad proměnnými, po každém přiřazení se znovu udržuje hranová konzistence. Tím se často odhalí spor dřív než hluboko ve stromu.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** CSP = proměnné, domény, podmínky, úplné konzistentní přiřazení. AC-3 škrtná hodnoty bez podpory u sousedních proměnných.
- 2 + k tomu: popiš frontu orientovaných hran v AC-3 a MAC jako backtracking plus opakované spuštění AC po každém přiřazení. Přidej mini příklad  $A < B$ .
- 3 **Bonus:** AC-3 pro binární podmínky má typicky složitost  $O(cd^3)$ , kde  $c$  je počet podmínek a  $d$  velikost domény. Hranová konzistence je jen lokální, proto sama negarantuje řešení. Forward checking po přiřazení čistí jen domény dosud nepřirazených sousedů, AC navíc udržuje kompatibilitu mezi nepřirazenými proměnnými. Obecnou  $n$ -ární podmínku lze převést na binární za cenu pomocných proměnných.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co přesně znamená, že hodnota má podporu v sousední doméně?
- Proč je hrana v CSP orientovaná, i když podmínka mezi proměnnými je binární?
- Jak se liší forward checking, AC-3 a MAC?
- Proč může být CSP hranově konzistentní a přesto nemít řešení?

**Pozor (častá past):** Nezaměň hranovou konzistenci s globální konzistencí. AC-3 škrtná zjevně nemožné hodnoty, ale nemusí najít celé přiřazení.

### 5.3 Logické uvažování

priorita: vysoká    cíl: zelená (2 body)

#### Logické uvažování

- základy výrokové logiky (konjunktivní a disjunktivní normální forma)
- algoritmus DPLL
- dopředné a zpětné řetězení (Hornovské klauzule)
- rezoluce

Učivo (jak na to):

Logické uvažování převádí znalosti na formule a otázku na test důsledku. Prakticky chceš umět dvě cesty: SAT přes DPLL a odvozování přes Hornovy klauzule nebo rezoluci.

CNF je konjunkce klauzulí, kde klauzule je disjunkce literálů. DNF je disjunkce konjunkcí literálů. Hornova klauzule má nejvýše jeden pozitivní literál, například  $\neg p \vee \neg q \vee r$ , což odpovídá pravidlu  $p \wedge q \Rightarrow r$ .

**DPLL** rozhoduje splnitelnost CNF. Opakuj jednotkovou propagaci, nastav čisté literály tak, aby splnily své klauzule, zkontroluj konec: žádné klauzule znamenají SAT, prázdná klauzule znamená UNSAT. Když není rozhodnuto, vyber proměnnou a větvi na pravda/nepravda.

**Řetězení:** dopředné řetězení jde od faktů k novým faktům, je data-driven. Zpětné řetězení jde od cíle k podcílům, je goal-driven. Obojí je levné a přirozené pro Hornovy klauzule.

**Rezoluce:** pro dotaz  $\alpha$  testuj nesplnitelnost  $KB \wedge \neg \alpha$  v CNF. Z klauzulí  $(A \vee p)$  a  $(B \vee \neg p)$  odvoď  $(A \vee B)$ . Když vznikne prázdná klauzule,  $KB \models \alpha$ .

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** vysvětli CNF, DNF, klauzuli, literál, Hornovu klauzuli a že důsledek lze testovat přes  $KB \wedge \neg\alpha$ .
- 2 + k tomu: popiš kroky DPLL: jednotková propagace, čistý literál, koncové stavy, větvení. Uveď rozdíl mezi dopředným a zpětným řetězením.
- 3 **Bonus:** napiš rezoluční pravidlo a vysvětli rezoluční zamítnutí. DPLL je v nejhorším exponenciální, ale jednotková propagace drasticky zmenšuje praxi. Dopředné řetězení lze implementovat počítadly nesplněných předpokladů pravidel; zpětné řetězení může být výrazně levnější, když dotaz míří jen na malou část báze znalostí.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co udělá DPLL s jednotkovou klauzulí?
- Jak přepíšeš  $\neg p \vee \neg q \vee r$  jako pravidlo?
- Proč se při důkazu dotazu přidává  $\neg\alpha$ ?
- Kdy je výhodnější zpětné řetězení než dopředné?

**Pozor (častá past):** Nepleť DPLL a rezoluci: DPLL hledá ohodnocení CNF, rezoluce odvozuje nové klauzule a hledá prázdnou klauzuli.

## 5.4 Pravděpodobnostní uvažování

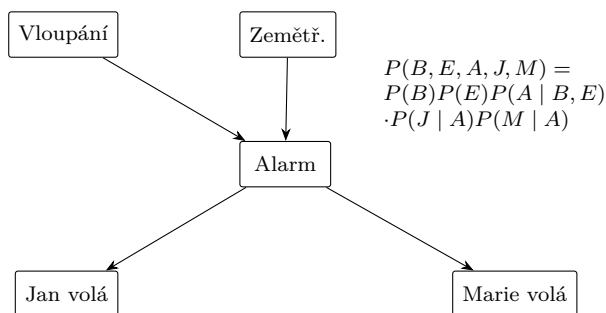
priorita: vysoká    cíl: zelená (2 body)

### Pravděpodobnostní uvažování

- základy teorie pravděpodobnosti (úplná sdružená distribuce, nezávislost, Bayesovo pravidlo)
- pravděpodobnostní uvažování (vysčítání, normalizace)
- Bayesovské sítě
- konstrukce a vztah k úplné sdružené distribuci
- exaktní odvozování (enumerace, eliminace proměnných)
- aproximační odvozování (Monte Carlo, zamítání, vážení věrohodností)

Učivo (jak na to):

Pravděpodobnost je způsob, jak uvažovat při nejistotě. Úplná sdružená distribuce umí odpovědět na všechno, ale je obrovská. Bayesovská síť ji komprimuje pomocí podmíněných nezávislostí.



Bayesovská síť je orientovaný acyklický graf náhodných proměnných a tabulky  $P(X_i |$

$Parents(X_i)$ ). Společná distribuce se faktorizuje jako

$$P(x_1, \dots, x_n) = \prod_i P(x_i \mid parents(x_i)).$$

**Základní počty:** z úplné sdružené distribuce získáš marginál vysčítáním skrytých proměnných, například  $P(X) = \sum_y P(X, y)$ . Podmíněnou pravděpodobnost získáš normalizací:  $P(X \mid e) = \alpha P(X, e)$ . Bayesovo pravidlo je  $P(H \mid e) = \alpha P(e \mid H)P(H)$ .

**Inference v síti:** enumerace přímo počítá přes skryté proměnné. Eliminace proměnných průběžně tvoří faktory a počítá jen jednou, takže šetří opakovanou práci. Aproximace generuje vzorky: Monte Carlo obecně, zamítání zahodí vzorky neslučitelné s evidencí, vážení věrohodností vzorky nezahazuje, ale váží je pravděpodobnostmi evidence.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** řekni úplná sdružená distribuce, nezávislost, Bayesovo pravidlo, vysčítání a normalizace. Bayesovská síť = DAG plus podmíněné tabulky.
- 2 **+ k tomu:** napiš faktorizaci sítě a ukaž ji na malé síti. Popiš enumeraci, eliminaci proměnných a rozdíl mezi zamítáním a vážením věrohodností.
- 3 **Bonus:** vysvětli, že konstrukce sítě závisí na pořadí proměnných a nejlepší je obvykle směr od příčin k důsledkům. Eliminace závisí na pořadí eliminovaných proměnných. Chain rule rozkládá sdruženou distribuci na součin podmíněných pravděpodobností. Naivní Bayes předpokládá podmíněnou nezávislost důsledků při známé příčině. Likelihood weighting fixuje evidenci a váží vzorky pravděpodobnostmi evidence.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak dostaneš  $P(X \mid e)$  z  $P(X, e)$ ?
- Co přesně faktorizuje Bayesovská síť?
- Proč je eliminace proměnných úspornější než čistá enumerace?
- Jaký je rozdíl mezi zamítáním a vážením věrohodností?

**Pozor (častá past):** Neříkej, že chybějící hrana znamená obyčejnou nezávislost. V Bayesovské síti jde hlavně o podmíněnou nezávislost vzhledem k rodičům a evidenci.

## 5.5 Reprezentace znalostí

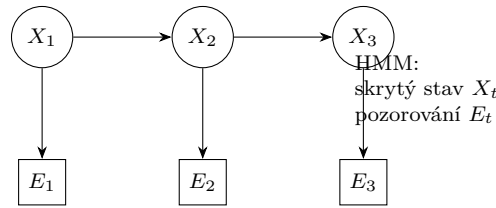
priorita: vysoká      cíl: zelená (2 body)

### Reprezentace znalostí

- situační kalkulus, problém rámce
- Markovské modely
- filtrace, predikce, vyhlazování, nejpravděpodobnější průchod
- skryté Markovské modely (HMM) vs. dynamické Bayesovské sítě

Učivo (jak na to):

Reprezentace znalostí řeší, jak zapsat svět tak, aby nad ním šlo odvozovat. Situační kalkulus je logický zápis změn po akcích, Markovské modely jsou pravděpodobnostní zápis vývoje stavu v čase.



Situační kalkulus popisuje situace vzniklé posloupností akcí, predikáty o situacích a akci  $Result(s, a)$ . Problém rámce je otázka, jak stručně zapsat, co se akcí nemění. HMM má skrytý stav  $X_t$ , pozorování  $E_t$ , přechod  $P(X_t | X_{t-1})$  a senzorový model  $P(E_t | X_t)$ .

**Markovské úlohy:** filtrace počítá  $P(X_t | e_{1:t})$ , tedy aktuální stav po dosavadních pozorováních. Predikce počítá budoucí stav  $P(X_{t+k} | e_{1:t})$ . Vyhlazování zpřesňuje minulý stav  $P(X_k | e_{1:t})$  pro  $k < t$ . Nejpravděpodobnější průchod hledá nejpravděpodobnější posloupnost skrytých stavů, typicky Viterbiho algoritmem.

**HMM vs. dynamická Bayesovská síť:** HMM má jednu skrytou stavovou proměnnou na čas. Dynamická Bayesovská síť rozkládá stav na více proměnných a dovolí bohatší závislosti mezi nimi.

**Odpověď podle bodů (cíl pro průchod: zelená = 2 body)**

- 1 **Minimum:** situační kalkulus = logika akcí a situací, problém rámce = jak nevyjmenovat všechny nezměněné fakty. HMM = skrytý stav, pozorování, přechodový a senzorový model.
- 2 **+ k tomu:** rozliš filtraci, predikci, vyhlazování a nejpravděpodobnější průchod. Nakresli řetěz  $X_1 \rightarrow X_2 \rightarrow X_3$  s pozorováními  $E_t$ .
- 3 **Bonus:** uveď, že HMM je speciální případ dynamické Bayesovské sítě; DBN dovolí faktorizovat stav na více proměnných a tím zmenšit tabulky. Markovův předpoklad říká, že další stav závisí jen na předchozím; stacionarita znamená stejné přechodové tabulky v čase. Filtrace používá forward algoritmus, vyhlazování forward-backward a nejpravděpodobnější průchod Viterbiho algoritmus.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je problém rámce v jedné větě?
- Jaký dotaz odpovídá filtraci a jaký vyhlazování?
- Co hledá Viterbiho algoritmus?
- Proč je DBN obecnější než HMM?

**Pozor (častá past):** Nezaměň filtraci s vyhlazováním: filtrace odhaduje současnost z minulých pozorování, vyhlazování zpětně zpřesňuje minulost i pomocí pozdější evidence.

## 5.6 Automatické plánování

priorita: vysoká    cíl: zelená (2 body)

### Automatické plánování

- formulace plánovacího problému (definice operátoru)
- dopředné a zpětné plánování

*Učivo (jak na to):*

Plánování je prohledávání nad faktorizovaným stavem. Místo černé skříňky používáme fakta, předpoklady akcí a efekty akcí, takže lze rozumněji vybírat relevantní kroky.

Klasický plánovací problém má počáteční stav jako množinu faktů, cíl jako podmínku na fakta a operátory. Operátor má preconditions, tedy kdy ho lze použít, a effects, tedy co přidá nebo smaže ve stavu. Plán je posloupnost operátorů, která z počátečního stavu vyrobí stav splňující cíl.

Mini příklad: operátor  $Přesun(x, A, B)$  má předpoklady  $Na(x, A)$  a  $Volne(B)$ , efekty přidat  $Na(x, B)$  a smazat  $Na(x, A)$ . Dopředné plánování začíná v počátečním stavu a zkouší použitelné akce. Zpětné plánování začíná cílem a hledá akce, které by cíl mohly splnit, přičemž jejich předpoklady se stanou novými podcíli.

- **Dopředné:** jednoduché simulování akcí, ale může zkoušet mnoho irelevantních kroků.
- **Zpětné:** soustředí se na cíl, ale musí řešit, zda vybrané akce jsou vzájemně slučitelné.
- **Kdy použít:** dopředné je přirozené, když je málo použitelných akcí; zpětné, když cíl silně omezuje relevantní akce.

Odpověď podle bodů (cíle pro průchod: zelená = 2 body)

- 1 **Minimum:** plánovací problém = počáteční stav, cílová podmínka, operátory. Operátor = předpoklady a efekty. Plán = posloupnost akcí.
- 2 **+ k tomu:** vysvětlí dopředné a zpětné plánování na mini příkladu s přesunem objektu. Řekni, proč faktorizovaný stav umožní pracovat s relevancí akcí.
- 3 **Bonus:** uveď rozdíl proti obyčejnému prohledávání: stav není atomický, akce mají logickou strukturu a plánovač může využívat předpoklady a efekty místo slepé expanze. Plánování používá předpoklad uzavřeného světa; fluenty se akcemi mění, rigidní atomy ne. Akce je plně instanciováný operátor a zpětné plánování používá jen akce relevantní pro cíl. Situační kalkulus řeší změny přes possibility a successor-state axiomy.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké části má operátor?
- Co je stav v klasickém plánování?
- Jak se liší dopředné a zpětné plánování?
- Proč je plánování vhodné pro faktorizované stavy?

**Pozor (častá past):** Neodpovídej jen obecně "hledání cesty". U plánování musí zaznít preconditions, effects a cíl jako podmínka na fakta.

## 5.7 Markovské rozhodovací procesy (MDP)

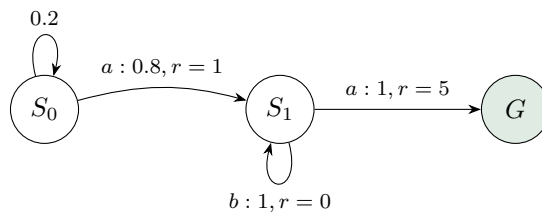
priorita: vysoká      cíl: zelená (2 body)

### Markovské rozhodovací procesy (MDP)

- formulace problému (výpočet užítku, strategie)
- Bellmanova rovnice
- iterace hodnot, iterace strategií
- POMDP (základní definice)

Učivo (jak na to):

MDP je plánování, kde akce nejsou jisté. Agent volí akci podle stavu, dostane odměnu a pravděpodobnostně přejde do dalšího stavu. Hledáme strategii s nejvyšším očekávaným diskontovaným užítkem.



MDP je čtveřice stavů, akcí, přechodových pravděpodobností  $P(s' | s, a)$  a odměn  $R(s, a, s')$ , často s diskontem  $\gamma$ . Strategie  $\pi(s)$  říká, jakou akci zvolit ve stavu  $s$ . Užitečnost stavu je očekávaný součet budoucích diskontovaných odměn.

**Bellmanova rovnice optimality:**

$$U(s) = \max_a \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma U(s')).$$

**Iterace hodnot:** opakovaně aktualizuj  $U(s)$  Bellmanovou rovnicí a nakonec z ní vyčti strategii.

**Iterace strategií:** střídá vyhodnocení současné strategie a její zlepšení výběrem lepší akce.

**POMDP:** stav není přímo pozorovaný, agent udržuje belief, tedy distribuci přes možné stavy.

**Odpověď podle bodů (cíl pro průchod: zelená = 2 body)**

- 1 **Minimum:** MDP = stavy, akce, přechody, odměny, strategie. Cíl je maximalizovat očekávaný diskontovaný užitek.
- 2 **+ k tomu:** napiš Bellmanovu rovnici a popiš iteraci hodnot i iteraci strategií. Uveď malý přechod se dvěma možnými následníky.
- 3 **Bonus:** POMDP = MDP s částečnou pozorovatelností, rozhoduje se nad belief stavem. Iterace strategií bývá dražší v kroku vyhodnocení, ale často potřebuje méně iterací. Diskontovaný užitek trajektorie je konečný pro  $\gamma < 1$  a omezené odměny. Optimální strategie závisí na aktuálním stavu, ne na počátečním. Policy evaluation řeší Bellmanovy rovnice bez maxima, policy improvement vybírá lepší akce.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je strategie v MDP?
- Kde v Bellmanově rovnici vystupuje nejistota?
- Jak se liší iterace hodnot a iterace strategií?
- Co je belief stav u POMDP?

**Pozor (častá past):** Nepleť MDP s HMM: v MDP agent volí akce a optimalizuje odměny, v HMM hlavně odhaduje skrytý stav z pozorování.

## 5.8 Hry a teorie her

**priorita: vysoká** **cíl: zelená (2 body)**

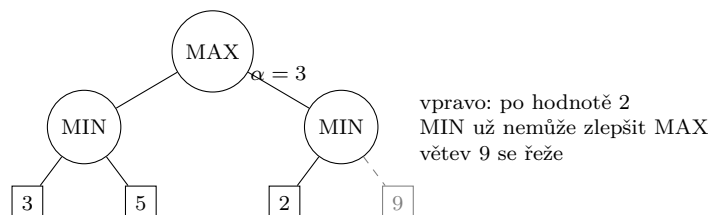
### Hry a teorie her

- algoritmy Minimax a alfa-beta prořezávání
- základy teorie her (věžňovo dilema, Nashovo ekvilibrium)
- mechanism design (typy aukcí)

*Učivo (jak na to):*



Ve hrách neoptimalizuješ proti přírodě, ale proti jinému agentovi. Minimax předpokládá, že soupeř hraje proti nám co nejlépe. Alfa-beta jen šetří práci, výsledek minimaxu nemění.



Minimax počítá hodnotu hry s nulovým součtem: v uzlu MAX bere maximum z potomků, v uzlu MIN minimum. Nashovo ekvilibrium je profil strategií, kde se žádnému hráči nevyplatí jednostranně změnit strategii.

**Alfa-beta:** drží  $\alpha$  jako nejlepší zatím zajištěnou hodnotu pro MAX a  $\beta$  jako nejlepší zatím zajištěnou hodnotu pro MIN. Když je jasné, že větev nemůže ovlivnit rozhodnutí předka, neprohledává se.

**Věžňovo dilema:** každý má individuální motivaci zradit, i když oboustranná spolupráce je společně lepší. Ukazuje rozdíl mezi individuální racionalitou a kolektivním výsledkem.

**Aukce:** anglická je vzestupná, holandská sestupná, první cena znamená platbu vlastní nabídky, druhá cena typicky motivuje nabídnout skutečnou hodnotu.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** minimax = MAX maximalizuje, MIN minimalizuje. Alfa-beta prořezává větve bez změny výsledku. Nash = žádná jednostranně výhodná odchylka.
- 2 **+ k tomu:** ukaž výpočet malého stromu a jeden alfa-beta řez. Vysvětli věžňovo dilema a vyjmenuj základní typy aukcí.
- 3 **Bonus:** dobré pořadí tahů dává alfa-beta velkou úsporu, v ideálním případě zhruba na efektivní hloubku poloviny stromu. U aukce druhé ceny zmiň pravdivé nabízení jako dominantní strategii v jednoduchém modelu. V hrách v normálním tvaru rozliš čisté a smíšené strategie, dominantní strategii, Pareto optimalitu a Nashovo ekvilibrium. Vickreyho aukce je obálková aukce druhé ceny; VCG zobecňuje pravdivé mechanismy pro víceparametrické aukce.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak se počítá hodnota v MAX a MIN uzlu?
- Proč alfa-beta nemění výsledek minimaxu?
- Co znamená Nashovo ekvilibrium?
- Jaký je rozdíl mezi aukcí první a druhé ceny?

**Pozor (častá past):** Neříkej, že alfa-beta je jiná strategie hraní. Je to optimalizace výpočtu stejného minimaxového rozhodnutí.

## 5.9 Strojové učení (přehledově)

priorita: vysoká    cíl: zelená (2 body)

### Strojové učení

- základní druhy učení (s učitelem, bez učitele, zpětnovazební)
- rozhodovací stromy (definice, konstrukce)

- regrese, SVM (základní principy)
- Bayesovské učení, EM algoritmus
- zpětnovazební učení
- pasivní učení (definice, metody ADP a TD)
- aktivní učení (definice, explorace vs. exploitace, Q-učení, SARSA)

*Učivo (jak na to):*

Tady se nečeká detail celé specializace Strojové učení, ale mapa pojmů v kontextu UI. Bezpečná odpověď vždy řekne typ úlohy, co se učí, z jakých dat a jak se pozná dobrý výsledek.

Učení s učitelem má dvojice vstup plus správný výstup a řeší klasifikaci nebo regresi. Učení bez učitele hledá strukturu bez cílových štítků. Zpětnovazební učení má agenta, akce, odměny a učí strategii z interakce s prostředím.

**Rozhodovací strom:** vnitřní uzel testuje atribut, hrana odpovídá výsledku testu a list dává predikci. Konstrukce vybírá test, který nejlépe rozdělí data, typicky podle informačního zisku nebo Gini nečistoty, a zastavuje nebo prořezává kvůli přeučení.

**Regrese a SVM:** regrese predikuje spojitou hodnotu, typicky minimalizuje chybu. SVM hledá oddělující nadrovinu s maximálním marginem; pro neseparabilní data používá měkký margin a pro nelineární hranice jádra.

**Bayesovské učení a EM:** Bayesovské učení kombinuje prior a věrohodnost na posterior  $P(h \mid data)$ . EM řeší skryté proměnné střídáním E-kroku, který dopočítá očekávání skrytých přiřazení, a M-kroku, který zlepší parametry.

**Zpětnovazební učení:** pasivní učení hodnotí pevně danou strategii. ADP odhaduje model přechodů a odměn a pak řeší MDP. TD aktualizuje hodnoty z rozdílu mezi po sobě jdoucími odhady. Aktivní učení současně volí akce: musí vyvažovat exploraci a exploitaci. Q-učení je off-policy aktualizace hodnot akcí, SARSA je on-policy aktualizace podle skutečně provedeného přechodu  $(s, a, r, s', a')$ .

**Odpověď podle bodů (cíl pro průchod: zelená = 2 body)**

- 1 **Minimum:** rozliš učení s učitelem, bez učitele a zpětnovazební. Definuj rozhodovací strom, regresi, SVM a RL slovníkem vstup, výstup, cíl.
- 2 **+ k tomu:** popiš konstrukci stromu, princip marginu u SVM, Bayesovské učení přes posterior, EM jako E/M kroky, pasivní ADP/TD a aktivní Q-učení/SARSA.
- 3 **Bonus:** zdůrazni rozdíl ADP vs. TD: ADP se učí model a řeší ho, TD se učí hodnoty přímo. Q-učení je off-policy s maximem přes další akce, SARSA je on-policy a používá další skutečně zvolenou akci. Přímý odhad utility průměruje reward-to-go, ale ignoruje Bellmanovy vazby. TD update:  $U(s) \leftarrow U(s) + \alpha(R(s) + \gamma U(s') - U(s))$ . Pro aktivní RL je exploration nutná, protože hladový agent může zůstat u špatně naučené akce;  $Q(s, a)$  dovolí volit akci bez znalosti přechodového modelu.

**Vyzkoušej se (zakryj text a odpověz nahlas)**

- Jak rozlišíš klasifikaci a regresi?
- Co je list a co vnitřní uzel v rozhodovacím stromu?
- Co se děje v E-kroku a M-kroku EM algoritmu?
- Jaký je rozdíl mezi Q-učením a SARSA?

**Pozor (častá past):** Nezajížděj mimo oficiální přehled: žádné hluboké učení, NLP ani robotika. Tady stačí základní principy uvedených metod.

## 6 Část IV: Specializace - Strojové učení

Toto je **nejdůležitější** okruh: vlastní zaměření a vázaná podmínka (ze specializace potřebuješ aspoň 7/12).  
Cíl: každé téma aspoň *zeleně* (2 body), nejdůležitější modely (lineární/logistická regrese, SVM, stromy, k-means, PCA) i *modře*.

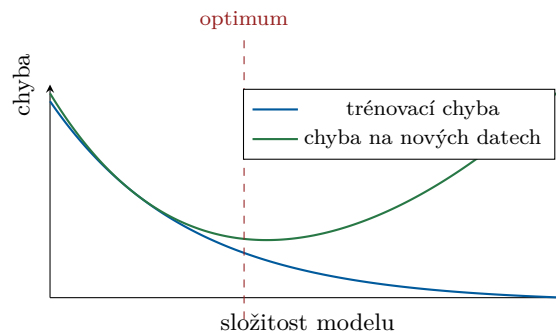
### 6.1 Učení s učitelem (základní pojmy, evaluace, regularizace)

priorita: vysoká      cíl: modrá (3 body) - rámec pro celý okruh

**Učení s učitelem:** klasifikace a regrese jako základní typy úloh; standardní evaluační metriky; ohodnocení modelu (testovací data, křížová validace, maximální věrohodnost); přeučení a regularizace (generalizační chyba, včasné zastavení trénování, L2 a L1 regularizace); prokletí dimenzionality.

*Učivo (jak na to):*

Z trénovacích dvojic (vstup, správný výstup) hledáme funkci, která dobře předpoví výstup i pro **nová** data. Nejde o to namemorovat trénink, ale *generalizovat*.



Příliš jednoduchý model = podtrénování; příliš složitý = přeučení. Hledáme dno zelené křivky.

**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

- 1 Minimum:** *klasifikace* = výstup je třída, *regrese* = výstup je číslo. Model trénujeme na trénovacích datech a hodnotíme na *oddělených testovacích* datech.
- 2 + k tomu:** *Generalizační chyba* = chyba na nových datech. *Přeučení* = model sedí na šumu trénovacích dat (malá trénovací, velká testovací chyba). Bráníme se: *regularizací* (L2 = penalizuje velké váhy  $\lambda \|w\|_2^2$ , hladší model; L1 =  $\lambda \|w\|_1$ , navíc dělá řídké váhy, tj. výběr příznaků), *včasným zastavením* trénování a více daty. *Křížová validace* (k-fold): data rozdělím na  $k$  částí, postupně každou použiju jako test, průměruju - spolehlivější odhad chyby než jediné rozdělení.
- 3 Bonus:** *metriky* - klasifikace: accuracy, precision, recall, F1 (z matice záměn); regrese: MSE/RMSE. *Maximální věrohodnost (MLE)*: parametry volíme tak, aby data byla nejpravděpodobnější; minimalizace záporné log-věrohodnosti je často přesně ta loss funkce. *Prokletí dimenzionality*: v mnoha rozměrech jsou data řídká a vzdálenosti ztrácejí smysl, je třeba exponenciálně víc dat. NLL je běžná klasifikační loss pro pravděpodobnostní modely. ROC mění práh a kreslí TPR proti FPR, AUC je plocha pod ní; senzitivita = recall, specificita = true negative rate. Hyperparametry se ladí na validační sadě, ne na testu. Redukce příznaků: filter je nezávislý na modelu, wrapper vybírá podle výkonu modelu.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak poznám přeučení z trénovací a testovací chyby?
- Jaký je rozdíl mezi L1 a L2 regularizací?
- K čemu je k-fold křížová validace a proč je lepší než jedno rozdělení?
- Co znamená precision vs recall?

**Pozor (častá past):** Model se nikdy nelaď (ani nevybírám) podle testovacích dat - na to je validační sada. Vysoká accuracy u nevyvážených tříd klame (proto F1).

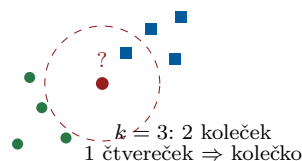
## 6.2 Učení založené na příkladech - k nejbližších sousedů (kNN)

priorita: střední    cíl: zelená (2 body)

**Učení založené na příkladech:** algoritmus k-nejbližších sousedů.

*Učivo (jak na to):*

„Řekni mi, kdo jsou tvoji sousedé, a já ti řeknu, kdo jsi.“ Nový bod dostane třídu podle většiny svých  $k$  nejbližších trénovacích bodů. Nic se netrénuje - jen se pamatují data.



**Odpověď podle bodů** (cíle pro průchod: zelená = 2 body)

- 1 **Minimum:** Pro nový bod najdi  $k$  nejbližších trénovacích bodů (podle vzdálenosti, typicky euklidovské) a přiřaď *většinovou* třídu (u regrese průměr).
- 2 **+ k tomu:** Žádné trénování (líné učení), veškerá práce je při dotazu. Malé  $k$  = citlivé na šum (přeučení), velké  $k$  = hladší, ale rozmazává hranice.  $k$  se volí křížovou validací. Nutné škálovat příznaky.
- 3 **Bonus:** Trpí prokletím dimenzionality a je pomalé při mnoha datech (dotaz prochází vše). Váhy sousedů mohou být uniformní, nepřímo úměrné vzdálenosti nebo softmax ze záporných vzdáleností; vzdálenosti se často počítají  $p$ -normou.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co se stane při  $k = 1$  a co při  $k =$  počet všech bodů?
- Proč je nutné příznaky normalizovat?

**Pozor (častá past):** kNN nic „nenatrénuje“ - náročný je až dotaz. A bez škálování dominuje příznak s velkým rozsahem.

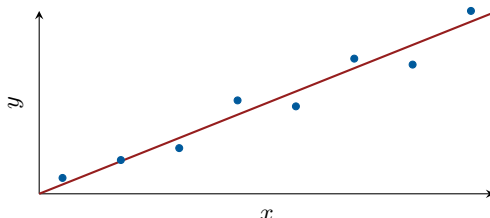
## 6.3 Lineární regrese

priorita: vysoká    cíl: modrá (3 body)

**Lineární regrese:** analytické řešení metodou nejmenších čtverců; trénování pomocí stochastic gradient descent.

*Učivo (jak na to):*

Proložíme daty **přímku** (obecně nadrovinu) tak, aby součet *čtverců* svislých odchylek byl co nejmenší.



**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

- ❶ **Minimum:** Model  $y = w^\top x + b$ . Minimalizujeme  $MSE \sum_i (y_i - w^\top x_i)^2$  (metoda nejmenších čtverců).
- ❷ **+ k tomu:** *Analytické řešení* (normální rovnice):  $\hat{w} = (X^\top X)^{-1} X^\top y$ , kde sloupec jedniček v  $X$  pokryje intercept  $b$ . Funguje, dokud  $X^\top X$  není moc velká / singulární.
- ❸ **Bonus:** *SGD* - místo přesného vzorce iterativně:  $w \leftarrow w - \eta \nabla_w \text{loss}$  počítané po malých dávkách dat; nutné u velkých dat.  $\eta$  = rychlost učení (moc velká diverguje, moc malá je pomalá). Batch GD používá všechna data, online SGD jeden řádek, minibatch kompromis; bias se pokryje sloupcem jedniček.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak zní normální rovnice a kdy ji nelze (rozumně) použít?
- Co řídí parametr  $\eta$  u SGD?

**Pozor (častá past):** Nejmenší čtverce trestají odlehlé body kvadraticky - jsou citlivé na outliery.

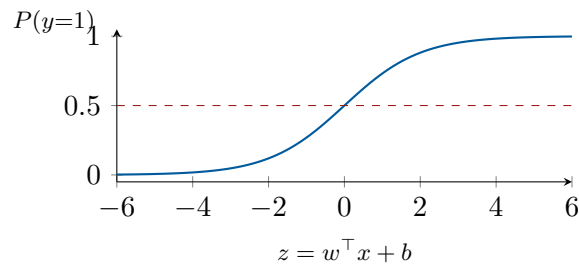
## 6.4 Logistická regrese

priorita: vysoká    cíl: modrá (3 body)

**Logistická regrese:** binární klasifikace (sigmoid, metody trénování); klasifikace do více tříd (softmax, metody trénování).

*Učivo (jak na to):*

Navzdory názvu je to **klasifikátor**: lineární skóre  $w^\top x$  protlačíme funkcí sigmoid, aby vyšla *pravděpodobnost* třídy mezi 0 a 1.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- ❶ **Minimum:**  $P(y=1 | x) = \sigma(w^T x + b)$ , kde  $\sigma(z) = \frac{1}{1+e^{-z}}$ . Klasifikuji podle prahu 0,5.
- ❷ **+ k tomu:** Trénuje se maximalizací věrohodnosti = minimalizací *cross-entropy* (*log-loss*) gradientním sestupem (analytické řešení neexistuje). Hranice je lineární.
- ❸ **Bonus:** Více tříd přes *softmax*  $P(y=k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$  (zobecnění sigmoidu); loss opět cross-entropy. Softmax je invariantní k přičtení konstanty ke všem skóre a binární sigmoid je jeho speciální případ. Lineární softmax má konvexní rozhodovací oblasti.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Proč se nepoužívá MSE, ale cross-entropy?
- Čím se liší sigmoid a softmax?

**Pozor (častá past):** Logistická regrese je klasifikace, ne regrese. Hranice je lineární (na nelineární potřebuješ příznaky navíc nebo jádro).

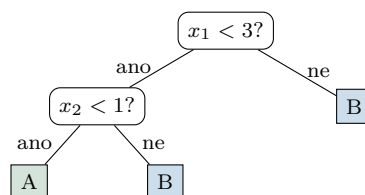
## 6.5 Rozhodovací stromy

priorita: vysoká      cíl: zelená (2 body)

**Rozhodovací stromy:** algoritmus učení a kritéria větvení; prořezávání.

Učivo (jak na to):

Posloupnost otázek typu „je příznak > práh?“. Každá otázka rozdělí data; chceme otázky, po kterých jsou výsledné skupiny co *nejčistší* (jedna třída).



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- ❶ **Minimum:** Strom dělí data podmínkami na příznacích; v listech je predikce (třída/hodnota). Staví se hladově shora.
- ❷ **+ k tomu:** V každém uzlu vyber dělení, které nejvíc *zvýší čistotu* - kritéria: *informační zisk* (pokles *entropie*) nebo *Gini index*. Rekurzivně až do čistých listů / zastavovacího kritéria.
- ❸ **Bonus:** Plně narostlý strom se přeučí  $\Rightarrow$  *prořezávání* (pre-pruning: zastav brzy podle hloubky/min. vzorků; post-pruning: ořež větev, které nezlepší chybu na validaci). Výhoda: interpretovatelnost. CART dělí list na dva a vybírá split s největším poklesem kritéria; pro regresi se používá squared error a list predikuje průměr. Reduced error pruning zruší větev,

pokud tím neklesne validační přesnost.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co měří entropie / Gini a co je informační zisk?
- Proč a jak se strom prořezává?

**Pozor (častá past):** Hluboký strom se skoro vždy přeučí. Stromy také snadno preferují příznaky s mnoha hodnotami.

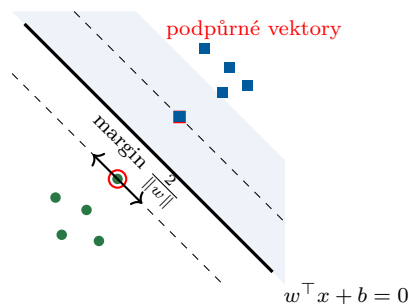
## 6.6 Metoda podpůrných vektorů (SVM)

priorita: vysoká      cíl: zelená (2 body)

**Metoda podpůrných vektorů:** klasifikátor pro lineárně separabilní třídy; klasifikátor pro lineárně neseperabilní třídy; jádrové funkce; klasifikace do více tříd.

Učivo (jak na to):

Mezi dvě třídy chceme proložit **co nejširší ulici** (pruh bez bodů). Hranice vede středem. Širší ulice = lepší generalizace.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** Najdi nadrovinu oddělující třídy s *největším marginem*; klasifikace  $\text{sign}(w^\top x + b)$ .
- 2 **+ k tomu:** *Podpůrné vektory* = body na okraji marginu (u měkkého marginu i uvnitř marginu / chybně klasifikované), jen ty určují hranici. Neseperabilní data: *měkký margin* se slack  $\xi_i$  a penaltou  $C \sum \xi_i$  ( $C$  = kompromis úzký margin vs. chyby). *Jádro*  $K(x, x')$  nahradí skalární součin a oddělí data ve vyšší dimenzi bez explicitní transformace (RBF, polynom).
- 3 **Bonus:** úloha  $\min \frac{1}{2} \|w\|^2$  za  $y_i(w^\top x_i + b) \geq 1$ ; více tříd přes one-vs-rest nebo one-vs-one. V duálu je  $w$  lineární kombinace trénovacích bodů a predikce má tvar  $\sum_i a_i t_i K(x, x_i) + b$ . Soft-margin používá slack proměnné  $\xi_i$  a hinge loss  $\max(0, 1 - t_i y(x_i))$ ; větší  $C$  znamená slabší regularizaci. RBF kernel  $e^{-\gamma \|x - z\|^2}$  dává vysokou podobnost blízkým bodům.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co jsou podpůrné vektory a co dělá parametr  $C$ ?
- Proč jádro umožní oddělit neseperabilní data?

**Pozor (častá past):** Velké  $C$  není „lepší“ - úzký margin se přeučí. Jádro nemění SVM, jen skalární součin.

## 6.7 Kombinace více modelů (ensemble)

priorita: vysoká      cíl: zelená (2 body)

**Kombinace více modelů:** bagging a boosting; metoda náhodných lesů.

*Učivo (jak na to):*

Více slabších modelů dohromady často předčí jeden velký - „moudrost davu“. Liší se tím, *jak* je spojíme.

**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

- 1 Minimum:** *Bagging* trénuje modely *paralelně* na náhodných podvýběrech dat (bootstrap) a *průměruje/hlasuje* - snižuje *rozptyl*. *Boosting* staví modely *sekvenčně*, každý opravuje chyby předchozích - často snižuje *bias*, ale je citlivější na šum.
- 2 + k tomu:** *Náhodný les* = bagging rozhodovacích stromů + v každém uzlu se uvažuje jen náhodná podmnožina příznaků (dekoreluje stromy). Robustní, málo ladění.
- 3 Bonus:** Boosting (např. AdaBoost) zvyšuje váhu špatně klasifikovaných příkladů; silný, ale citlivější na šum/přeučení než bagging. AdaBoost váží i slabé klasifikátory podle přesnosti. Náhodný les při regresi průměruje stromy a při klasifikaci hlasuje.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Bagging vs boosting: paralelně/sekvenčně, snižuje rozptyl/bias?
- Čím se náhodný les liší od prostého baggingu stromů?

**Pozor (častá past):** Boosting nezaměňuj s baggingem: boosting je sekvenční a typicky cílí hlavně na bias, bagging paralelní na rozptyl.

## 6.8 Statistické testy

priorita: střední      cíl: zelená (2 body)

**Statistické testy:** studentův t-test (jednovýběrový a dvouvýběrový); chí-kvadrát test (test dobré shody).

*Učivo (jak na to):*

Ptáme se: je pozorovaný rozdíl **reálný**, nebo jen náhoda? Spočteme testovou statistiku a porovnáme s tím, co by dělala náhoda za platnosti *nulové hypotézy*.

**Odpověď podle bodů** (cíl pro průchod: zelená = 2 body)

- 1 Minimum:** *Nulová hypotéza*  $H_0$  = „žádný efekt“. Test dá *p-hodnotu*; když  $p < \alpha$  (typicky 0,05),  $H_0$  zamítáme.
- 2 + k tomu:** *t-test* porovnává *střední hodnoty*: jednovýběrový (průměr vs. daná konstanta), dvouvýběrový (dva průměry proti sobě); předpokládá zhruba normální data. *Chí-kvadrát test dobré shody* porovnává *pozorované vs. očekávané četnosti* kategorií:  $\chi^2 = \sum \frac{(O-E)^2}{E}$ .
- 3 Bonus:** chyba I. druhu = zamítnu platné  $H_0$  ( $= \alpha$ ); chyba II. druhu = nezamítnu neplatné  $H_0$ . Jednovýběrový t-test používá  $n - 1$  stupňů volnosti, dvouvýběrový porovnává průměry



dvou populací přes směrodatnou chybu; párový t-test testuje rozdíly dvojic jako jednovýběrový. Chí-kvadrát dobré shody má obvykle počet kategorií minus jedna stupňů volnosti.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je p-hodnota a kdy zamítám  $H_0$ ?
- Kdy t-test a kdy chí-kvadrát?

**Pozor (častá past):** Malá p-hodnota neznamena „velký efekt“, jen „těžko vysvětlitelný náhodou“.

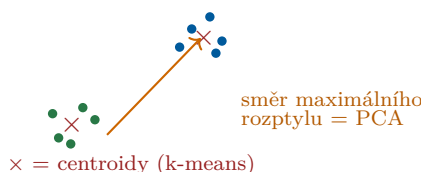
## 6.9 Učení bez učitele (shlukování, PCA)

priorita: vysoká    cíl: zelená (2 body)

**Učení bez učitele:** shlukování (algoritmus k-means); hierarchické shlukování; redukce dimenzionality (analýza hlavních komponent, PCA).

Učivo (jak na to):

Nemáme správné odpovědi, jen data. Hledáme v nich *strukturu*: přirozené skupiny (shlukování) nebo směry, kde se data nejvíc mění (PCA).



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** *Shlukování* = rozděl data do skupin podle podobnosti. *PCA* = najdi pár směrů, které zachytí nejvíc rozptylu, a do nich data promítni (zmenšení dimenze).
- 2 **+ k tomu:** *k-means*: zvol  $k$  center, opakuj {přiřaď body nejbližšímu centru; přepočítej centra jako průměry} až do ustálení. *Hierarchické shlukování*: postupně slučuj nejbližší shluky (aglomerativně)  $\Rightarrow$  *dendrogram*,  $k$  není třeba dopředu.
- 3 **Bonus:** *PCA* = vlastní vektory kovarianční matice (největší vlastní čísla = nejvíc rozptylu). *k-means*: nutné zvolit  $k$  a citlivý na počáteční centra (*k-means++*). *k-means* minimalizuje součet čtverců vzdáleností ke centroidům a konverguje k lokálnímu optimu. Hierarchické shlukování může být aglomerativní nebo divisivní, dendrogram se řeže podle požadovaného počtu shluků.  $SVD\ X = U\Sigma V^T$  dává nízkohodnostní aproximace; *PCA* centruje data a bere první sloupce  $V$ .

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké dva kroky k-means opakuje?
- Co maximalizují hlavní komponenty a jak souvisí s kovarianční maticí?
- Výhoda hierarchického shlukování oproti k-means?

**Pozor (častá past):** k-means hledá zhruba kulové, podobně velké shluky a vyžaduje předem  $k$ ; na protáhlé/různě husté shluky selhává.